

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ДОМАШНЯЯ РАБОТА

По дисциплине Программирование

Тема работы Программное обеспечение для работы с физическими измерениями

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3121

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Казанова П.П.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	4
1.1 Аналогии	4
1.2 Функциональные требования	4
1.3 Нефункциональные требования	5
1.4 Хранение записей	5
2 БИБЛИОТЕКА ДЛЯ РАБОТЫ С ИЗМЕРЕНИЯМИ	6
2.1 Структура	6
2.2 Простое измерение	7
2.3 Множественное измерение	7
2.4 Совместное измерение	8
2.5 Калькулятор косвенных измерений	8
2.6 Исключения и инициализация	8
3 ГРАФИЧЕСКОЕ ПРИЛОЖЕНИЕ	9
3.1 Диаграмма классов	9
3.2 База данных	10
3.3 Алгоритм работы приложения	10
ЗАКЛЮЧЕНИЕ	12
ПРИЛОЖЕНИЕ А Исходный код	13
ПРИЛОЖЕНИЕ Б Интерфейс	61

ВВЕДЕНИЕ

В отчёте представлено решение домашней работы: программный код на python для реализации технического задания и объяснение его работы. Отчёт, как и программа, состоит из двух частей: библиотека для работы с измерениями и приложение с графическим интерфейсом. В приложении Б представлен графический интерфейс, а в приложении А – листинги кода.

Техническое задание

Приложение с графическим интерфейсом, хранением данных в базе данных, обработкой ошибок, реализованное с помощью технологий ООП.

Цель

Разработать программное обеспечение для работы с физическими измерениями, удовлетворяющее техническому заданию.

Задачи

- 1) Разработать библиотеку для работы с измерениями.
- 2) Придумать и реализовать хранение результатов измерений в базе данных.
- 3) Реализовать приложение с графическим интерфейсом для работы с библиотекой и базой данных.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Программа предназначена для систематизации, анализа и визуализации экспериментальных данных. Ключевые задачи: автоматизация рутинных расчётов (погрешности, обработка формул, множественные измерения), упрощение статистической обработки (линейная зависимость по методу наименьших квадратов), визуализация данных через графики зависимостей. Всё это снижает риск ошибок, экономит время пользователей и улучшает точность научных работ.

1.1 Аналоги

OriginLab, GraphPad Prism — мощные инструменты, но требуют обучения и дороги для частных пользователей.

SciDAVis, GNU Octave — бесплатные аналоги с уклоном в научные расчёты, но имеют сложный интерфейс.

Physics Toolbox Suite (мобильное приложение) — удобен для быстрых измерений, но ограничен в анализе.

Пользователи ценят в аналогах точность расчётов, но критикуют избыточность функций и высокий порог входа.

В отличие от аналогов, разрабатываемое приложение заточено для работы по правилам из методических пособий ИТМО по физике.

1.2 Функциональные требования

- Создавать, удалять эксперименты, переключаться между ними.
- Добавлять, изменять, удалять измерения.
- Рассчитывать множественное измерение.
- Рассчитывать измерения по формуле.
- Рассчитывать измерения по линейной зависимости (МНК).
- Строить график линейной зависимости (МНК).

1.3 Нефункциональные требования

- понятность для пользователя,
- скорость работы,
- обработка ошибок.

1.4 Хранение записей

Результат измерения хранится в виде записи:

- уникальный номер,
- номер эксперимента,
- значение,
- абсолютная погрешность,
- относительная погрешность,
- метка прямого измерения,
- название,
- символ,
- единица измерения.

2 БИБЛИОТЕКА ДЛЯ РАБОТЫ С ИЗМЕРЕНИЯМИ

В этой главе описано создание библиотеки для работы с измерениями.

2.1 Структура

На рисунке 1 представлена диаграмма классов библиотеки.

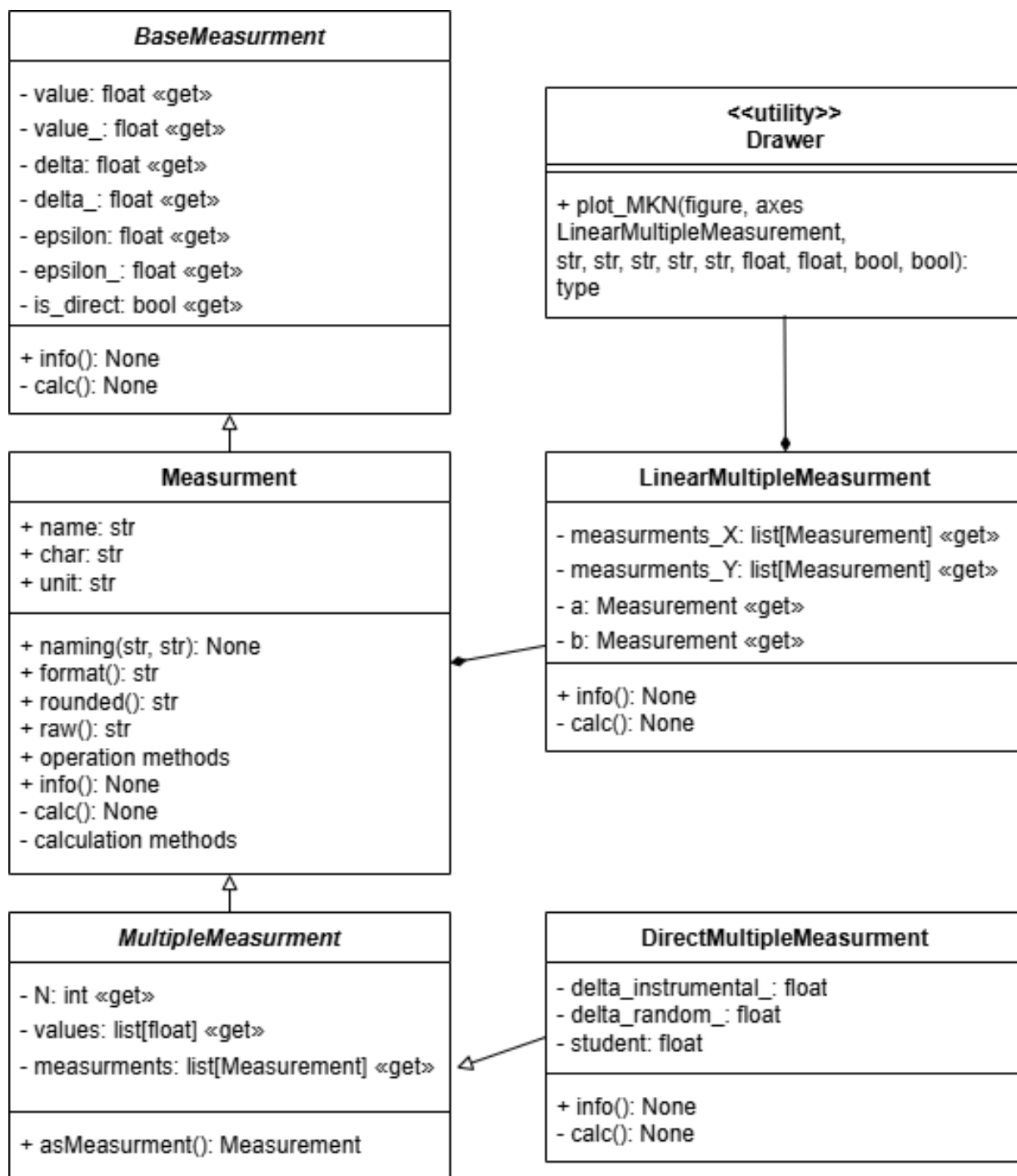


Рисунок 1 — Диаграмма классов библиотеки

2.2 Простое измерение

Абстрактный класс «BaseMeasurement» представлен на рисунках 4 - 5. В нём объявлены основные для измерения поля и 2 абстрактных метода: закрытый «_calc» для расчётов и «info» для вывода всех основных параметров (рисунок 5, строка 1).

Класс «Measurement», наследуемый от «BaseMeasurement» представлен на рисунках 6 - 10.

При инициализации записываются основные параметры, затем вызываются «_dimer» для домножения значений на параметр «dim» (рисунок 6, строка 43), «_calc» для расчётов относительной погрешности по абсолютной и наоборот (рисунок 6, строка 32) и, наконец округляется методом «_round» (рисунок 7, строка 19), использующим ещё несколько приватных методов.

Методом «naming» можно задать название и букву измерения (рисунок 7, строка 26).

После расчётов можно получить результаты различными методами. Метод «format» возвращает строку вида « $a \pm \Delta a$ » (рисунок 8, строка 2), методы «rounded» и «raw» возвращает округлённую и не округлённую строку вида « $a \pm \Delta a, \varepsilon_a$ » (рисунок 8, строка 36).

Также для измерения переопределены методы сложения, вычитания, умножения, деления, отрицания и взятия по модулю для некоторых случаев (рисунки 9 и 10).

2.3 Множественное измерение

Абстрактный класс «MultipleMeasurement» представлен на рисунках 11 - 12, он наследуется от обычного измерения. В нём объявлены дополнительные поля «values» и «measurments» и «N» и метод-параметр «asMeasurment» для преобразования множественного измерения в обычное (рисунок 12, строка 8).

Класс «DirectMultipleMeasurement», наследуемый от «MultipleMeasurement» представлен на рисунке 13.

В нём добавлены поля инструментальной погрешности «delta_instrumental», статистической погрешности «delta_random» и коэффициента Стьюдента «student». Переопределён метод «_calc» для расчётов (рисунок 13, строка 30).

2.4 Совместное измерение

Класс «LinearMultipleMeasurement», наследуемый от «MultipleMeasurement» представлен на рисунках 14 - 15.

Полями класса являются списки измерений «measurments_X» и «measurments_Y», коэффициенты линейной зависимости «a» и «b». Метод «_calc» определён для расчётов по методу наименьших квадратов (МНК) (рисунок 15, строка 40).

Класс «Drawer» (рисунки 16 - 19) используется для рисования графиков линейной зависимости методом «plot_MKN». Для рисования используется библиотека «matplotlib».

2.5 Калькулятор косвенных измерений

В файле «Expression» (рисунки 20 и 21) объявлена единственная функция «expression», которая принимает на вход все измерения, физическую формулу в виде строки и возвращает «Measurement» или выбрасывает исключение «ExpressionError». Для вычислений используется библиотека «sympy».

2.6 Исключения и инициализация

В файле «MeasException» (рисунок 22) объявлены 3 собственных исключения «MeasException» для общих ошибок, «StudentException» для ошибок, связанных с отсутствием коэффициента Стьюдента, «ExpressionError» для ошибок в результате работы функции expression.

Файл «__init__.py» (рисунок 23) нужен для удобного импорта библиотеки.

3 ГРАФИЧЕСКОЕ ПРИЛОЖЕНИЕ

3.1 Диаграмма классов

На рисунке 2 представлена диаграмма классов библиотеки.

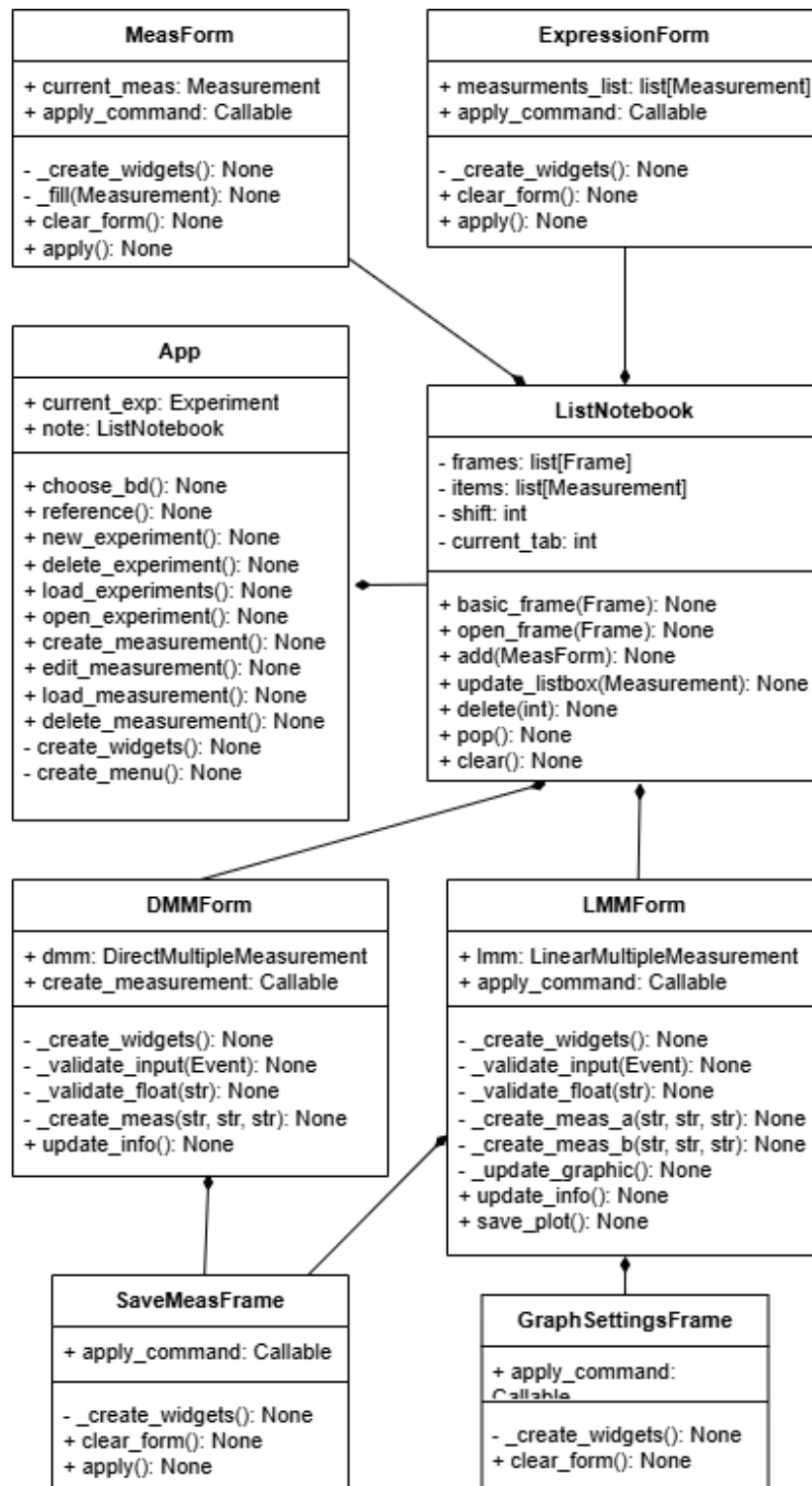


Рисунок 2 — Диаграмма классов библиотеки

3.2 База данных

Для хранения данных используется sqlite под управлением ORM peewee (рисунок 51). Даталогическая модель базы данных представлена на рисунке 3.

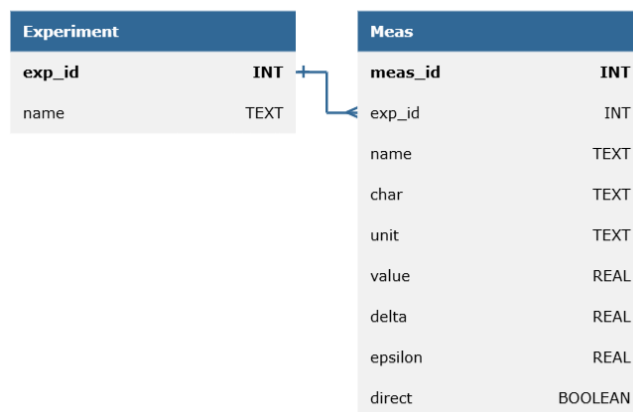


Рисунок 3 — Даталогическая модель базы данных

3.3 Алгоритм работы приложения

Графический интерфейс приложения основан на библиотеке «tkinter». Каждый класс, входящий в программу, представляет собой графический элемент.

Окно приложения – класс «App» (рисунки 42 - 43). В него входят меню, нижняя панель с кнопками и «ListNotebook». В этом же классе прописаны основные функции для взаимодействия с базой данных.

Класс «ListNotebook» наследуемый от «Frame» (рисунки 35 - 36). Он состоит из списка выбора измерения и фрейма с формами для работы.

Форма «MeasForm» нужен для добавления и редактирования измерений (рисунки 47 - 50).

Форма «ExpressionForm» для расчётных измерений (рисунки 29 - 31).

Форма «DMMForm» для работы с множественными измерениями (рисунки 25 - 27), её частью является фрейм для ввода имени, символа и единицы измерения «SaveMeasFrame» (рисунки 52 - 53).

Форма «LMMForm» для работы с совместными измерениями (рисунки 37 - 41), её частями является «SaveMeasFrame» и фрейм с настройкой графика «GraphicSettongsFrame» (рисунки 32 - 34).

Интерфейс программы представлен в приложении Б на рисунках 54 - 57.

ЗАКЛЮЧЕНИЕ

Цель практической работы достигнута. Представлено решение задачи с выполненными пунктами технического задания: приложение с графическим интерфейсом, хранением данных в базе данных, обработкой ошибок, реализованное с помощью технологий ООП. Были применены навыки поиска информации в интернете и программирования на python.

ПРИЛОЖЕНИЕ А

Исходный код

```
1 from abc import ABC, abstractmethod
2
3 class BaseMeasurement(ABC):
4     _value: float      # окр. значение
5     _value_: float     # значение
6     _delta: float      # окр. абсолютная
7     _delta_: float     # абсолютная
8     _epsilon: float    # окр. относительная
9     _epsilon_: float   # относительная
10
11     # Флаг прямого измерения, True - 1.00, False - 0.95
12     _is_direct: bool
13
14     def _idm(self):
15         return 2/3 if self._is_direct else 1
16
17     @property
18     def value(self):
19         return self._value
20
21     @property
22     def v(self):
23         return self._value_
24
25     @property
26     def value_(self):
27         return self._value_
28
29     @property
30     def delta(self):
31         return self._delta
32
33     @property
34     def delta_(self):
35         return self._delta_
36
37     @property
38     def epsilon(self):
39         return self._epsilon
40
41     @property
42     def epsilon_(self):
43         return self._epsilon_
44
45     @property
46     def is_direct(self):
47         return self._is_direct
48
49     @abstractmethod
50     def _calc(self):
```

Рисунок 4 — Код в файле «BaseMeasurement.py» часть 1

```
1         raise NotImplementedError('This method is not implemented!')
2
3     @abstractmethod
4     def info(self):
5         raise NotImplementedError('This method is not implemented!')
6
```

Рисунок 5 — Код в файле «BaseMeasurement.py» часть 2

```

1 from math import *
2 from .Tools import *
3 from .BaseMeasurement import *
4 from copy import deepcopy
5
6 class Measurement(BaseMeasurement, Tools):
7     def __init__(self, value:float,
8                  delta:float|None=None, epsilon:float|None=None,
9                  name:str|None=None, char:str='', unit:str='',
10                 dim:float|None=None, direct:bool=False):
11         if isinstance(value, str):
12             name = value
13
14         self.name = name
15         self.char = char
16         self.unit = unit
17         self._value_ = value
18         self._delta_ = delta
19         self._epsilon_ = epsilon
20         self._is_direct = direct
21         self._dimer(dim)
22         self._calc()
23         self._round()
24
25     def naming(self, char:str, unit:str=''):
26         self.char = char
27         self.unit = unit
28
29     def _soft(self, x):
30         # Округлить компьютерную погрешность
31         CALC_ERROR = 12
32         x = round(x, CALC_ERROR)
33         return 0 if isclose(x, 0) else x
34
35     def _calc(self):
36         if self._epsilon_ is None and self._delta_ is None:
37             self._epsilon_, self._delta_ = 0, 10-13
38         if self._epsilon_ is None:
39             if isclose(self._value_, 0):
40                 self._epsilon_ = 0
41             else:
42                 self._epsilon_ = (self._delta_ / abs(self._value_) * 100)
43         elif self._delta_ is None:
44             self._delta_ = self._epsilon_ * self._value_ / 100
45
46     def _dimer(self, dim):
47         if dim is not None:
48             self._value_ *= dim
49             if self._delta_ is not None:
50                 self._delta_ *= dim

```

Рисунок 6 — Код в файле «Measurement.py» часть 1

```

1
2
3 def _get_first_significant_digit(self, n):
4     # Получаем первую значащую цифру
5     n_abs = abs(float(n))
6     if n_abs == 0:
7         return 0
8     s = "{:.15e}".format(n_abs)
9     mantissa, exp = s.split('e')
10    mantissa = mantissa.replace('.', '').lstrip('0')
11    return int(mantissa[0]) if mantissa else 0
12
13 def _round_to_significant(self, value, significant):
14     # Округлить до значащей цифры
15     if value == 0:
16         return 0.0
17     return float("{:.{}e}".format(value, significant-1))
18
19 def _format_with_decimals(self, number, decimals):
20     # Форматируем число с фиксированным количеством знаков
21     return "{0:." + str(decimals) + "f}".format(number)
22
23 def _round(self):
24     # Обработка абсолютной погрешности
25     first_digit = self._get_first_significant_digit(self._delta_)
26     k_delta_x = 2 if first_digit in {1, 2, 3} else 1
27     rounded_delta_x = self._round_to_significant(self._delta_, k_delta_x)
28
29     # Определение порядка округления для x
30     formatted_delta = "{:.{prec}e}".format(rounded_delta_x, prec=k_delta_x-1)
31     mantissa, exp_part = formatted_delta.split('e')
32     exp = int(exp_part)
33     order = 10 ** (exp - (k_delta_x - 1))
34
35     if order != 0:
36         rounded_x = round(self._value_ / order) * order
37     else:
38         rounded_x = self._value_
39
40     # Обработка относительной погрешности
41     rounded_delta_rel = None
42
43     first_digit_rel = self._get_first_significant_digit(self._epsilon_)
44     k_delta_rel = 2 if first_digit_rel in {1, 2, 3} else 1
45     rounded_delta_rel = self._round_to_significant(self._epsilon_, k_delta_rel)
46
47     # ИЗМЕНЕНО! Добавлен soft на округление!
48     self._value = self._soft(rounded_x)
49     self._delta = self._soft(rounded_delta_x)
50     self._epsilon = self._soft(rounded_delta_rel)

```

Рисунок 7 — Код в файле «Measurement.py» часть 2


```

1  def format(self):
2      # Обработка абсолютной погрешности
3      first_digit = self._get_first_significant_digit(self._delta_)
4      k_delta = 2 if first_digit in {1, 2, 3} else 1
5      rounded_delta = self._round_to_significant(self._delta_, k_delta)
6
7      # Определяем порядок округления для x
8      delta_str = "{:.{}e}".format(rounded_delta, k_delta-1)
9      mantissa_part, exp_part = delta_str.split('e')
10     exponent = int(exp_part)
11     step = 10 ** (exponent - (k_delta - 1))
12
13     # Округляем x до нужного шага
14     rounded_x = round(self._value_ / step) * step
15
16     # Определяем количество знаков после запятой
17     if step < 1:
18         decimals = -int(log10(step))
19     else:
20         decimals = 0
21
22     # Форматируем с сохранением всех нулей
23     x_str = self._format_with_decimals(rounded_x, decimals)
24     delta_str = self._format_with_decimals(rounded_delta, decimals)
25
26     # Убираем лишние точки для целых чисел
27     if decimals == 0:
28         x_str = x_str.split('.')[0]
29         delta_str = delta_str.split('.')[0]
30
31     char_pm = '±'
32     return f"{x_str}{char_pm}{delta_str}"
33
34 @property
35 def rounded(self):
36     # Для дебага округлённых значений
37     return f"{self.format()}, ε = {self._epsilon}%"
38
39 @property
40 def raw(self):
41     # Для дебага сырых значений
42     return (f"{self._soft(self._value_)}, " +
43            f"Δ = {self._soft(self._delta_):.9f}," +
44            f"ε = {self._soft(self._epsilon):.9f}")
45
46 def __str__(self):
47     return f'{self.rounded} ({self.raw})'
48
49 def __repr__(self):
50     return f'{self.rounded}'

```

Рисунок 8 — Код в файле «Measurement.py» часть 3

```

1
2 def info(self):
3     return f'{self.rounded} ({self.raw})'
4
5 def __eq__(self, other): return self._value == other.value
6 def __ne__(self, other): return not (self == other)
7
8 def __gt__(self, other): return self._value > other.value
9 def __le__(self, other): return not (self > other)
10
11 def __lt__(self, other): return self._value < other.value
12 def __ge__(self, other): return not (self < other)
13
14
15 def __neg__(self):
16     new_self = deepcopy(self)
17     new_self._value_ = -self.value_
18     new_self._value = -self.value
19     return new_self
20
21 def __abs__(self):
22     new_self = deepcopy(self)
23     new_self._value_ = abs(self.value_)
24     new_self._value = abs(self.value)
25     return new_self
26
27 def __add__(self, other):
28     if isinstance(other, Measurement):
29         return Measurement(
30             self._value_ + other._value_,
31             sqrt((self._idm() * self._delta_)**2
32                 + (self._idm() * other._delta_)**2)
33         )
34     raise ValueError("Можно складывать только измерения")
35
36
37 def __sub__(self, other):
38     if isinstance(other, Measurement):
39         return Measurement(
40             self._value_ - other._value_,
41             sqrt((self._idm * self._delta_)**2
42                 + (self._idm * other._delta_)**2)
43         )
44     raise ValueError("Можно вычитать только измерения")
45
46 def __mul__(self, other):
47     if isinstance(other, (int, float)):
48         return Measurement(
49             self.value_ * other,
50             self.delta_ * other,

```

```

1         direct=self._is_direct
2     )
3
4     def __truediv__(self, other):
5         if isinstance(other, (int, float)):
6             if isclose(other, 0):
7                 raise ZeroDivisionError()
8             return Measurement(
9                 self.value_ / other,
10                self.delta_ / other,
11                direct=self._is_direct
12            )
13

```

Рисунок 10 — Код в файле «Measurement.py» часть 5

```

1 from .Measurement import Measurement
2 from .MeasException import *
3 from abc import ABC
4
5 student_koef = {2:12.7, 3:4.30, 4:3.18, 5:2.78, 6:2.57, 7:2.45,
6                8:2.36, 9:2.31, 10:2.26, 20:2.09, 30:2.04}
7
8 class MultipleMeasurement(Measurement, ABC):
9     _N: int
10    _values: list[float]
11    _measurments: list[Measurement]
12    _is_direct = False
13
14    @property
15    def values(self):
16        return self._values
17
18    @property
19    def measurments(self):
20        return self._measurments
21
22    @property
23    def N(self):
24        return self._N
25
26    def __init__(self, measured_values: list[int|float|Measurement],
27                dim: int|None=None,
28                name: str|None=None, char: str='', unit: str='',):
29        self._N = len(measured_values)
30        if self._N not in student_koef:
31            raise StudentException(self._N)
32        self._student = student_koef[self._N]
33        self._char = char
34        self._unit = unit
35        self._name = name
36        self._dimer(dim)
37        self._calc()
38        self._round()
39
40    def _dimer(self, dim):
41        if dim is not None:
42            self._values = [x * dim for x in self._values]
43            self._measurments = [
44                Measurement(x._value_, x._delta_, dim=dim)
45                for x in self._measurments
46            ]
47
48    def info(self):
49        return f"\nN = {self._N}\nvalues = {self._values}" \
50            f"\naverage = {self._value} ({self._value_})" \

```

Рисунок 11 — Код в файле «MultipleMeasurement.py» часть 1

```
1         f"\ndelta = {self._delta} ({self._delta_})" \
2         f"\nepsilon = {self._epsilon} ({self._epsilon_})"
3
4     def __getitem__(self, index):
5         return self._measurments[index]
6
7     @property
8     def asMeasurement(self) -> Measurement:
9         return Measurement(self._value, self._delta, self._epsilon,
10                            self._name, self._char, self._unit)
```

Рисунок 12 — Код в файле «MultipleMeasurement.py» часть 2

```

1 from math import *
2 from .MultipleMeasurement import *
3
4 class DirectMultipleMeasurement(MultipleMeasurement):
5     _delta_instrumental_: float
6     _delta_random_: float
7     _student: float
8
9     @property
10    def values(self):
11        return self._values
12
13    @property
14    def N(self):
15        return self._N
16
17    def __init__(self,
18        measured_values: list[int|float|Measurement],
19        delta_instrumental: int|float=0,
20        dim: int|None=None,
21        name: str|None=None, char: str='', unit: str=''):
22        self._delta_instrumental_ = delta_instrumental
23        if len(measured_values) == 0:
24            raise MeasException("Пустой список!")
25        if isinstance(measured_values[0], Measurement):
26            self._measurments = measured_values
27            self._values = [x.value_ for x in measured_values]
28        else:
29            self._values = measured_values
30            self._measurments = [
31                Measurement(x, delta_instrumental, direct=True)
32                for x in measured_values
33            ]
34        if dim is not None:
35            self._delta_instrumental_ *= dim
36        super().__init__(measured_values, dim=dim, name=name, char=char, unit=unit)
37
38    def _calc(self):
39        # Считаем значение и погрешность
40        self._value_ = sum(self._values) / self._N
41        self._sigma_ = sqrt(sum((x - self._value_)**2 for x in self._values)
42            /(self._N*(self._N - 1)))
43        self._delta_random_ = self._sigma_ * self._student
44        self._delta_ = sqrt(self._delta_random_**2
45            + (4/9)*self._delta_instrumental_**2)
46        if not isclose(self._value_, 0):
47            self._epsilon_ = self._delta_ / self._value_ * 100
48        else:
49            self._epsilon_ = 0
50

```

Рисунок 13 — Код в файле «DirectMultipleMeasurement.py» часть 1

```

1 from math import *
2 from .Measurement import *
3 from .MultipleMeasurement import *
4
5 class LinearMultipleMeasurement():
6     _measurments_X: list[Measurement]
7     _measurments_Y: list[Measurement]
8     _a: Measurement
9     _b: Measurement
10
11     @property
12     def a(self):
13         return self._a
14
15     @property
16     def b(self):
17         return self._b
18
19     @property
20     def measurments_X(self):
21         return self._measurments_X
22
23     @property
24     def measurments_Y(self):
25         return self._measurments_Y
26
27     @check_len
28     def __init__(self, measurments_X, measurments_Y, name=None):
29         self.name = name
30         self._N = len(measurments_X)
31         if isinstance(measurments_X[0], Measurement):
32             self._measurments_X = measurments_X
33             self._measurments_Y = measurments_Y
34         else:
35             self._measurments_X = [Measurement(x, 0) for x in measurments_X]
36             self._measurments_Y = [Measurement(y, 0) for y in measurments_Y]
37         self._calc()
38
39     @timer
40     def _calc(self):
41         N = self._N
42         x = [x.value_ for x in self._measurments_X]
43         y = [y.value_ for y in self._measurments_Y]
44         av_x = sum(x) / N
45         av_y = sum(y) / N
46         b = (sum((x[i] - av_x) * (y[i] - av_y) for i in range(N))
47              / sum((x[i] - av_x)**2 for i in range(N)))
48         a = av_y - b * av_x
49         d = [y[i] - (a + b*x[i]) for i in range(N)]
50         D = sum((x[i] - av_x)**2 for i in range(N))

```

Рисунок 14 — Код в файле «LinearMultipleMeasurement.py» часть 1

```

1      # print(f'av_x = {av_x}, D = {D}')
2      sum_sQ = (sum(d[i]**2 for i in range(N))) / (N - 2)
3      sigma_bQ = sum_sQ / D
4      sigma_aQ = (1/N + av_x**2/D) * (sum_sQ)
5      delta_a = 2*sqrt(sigma_aQ)
6      delta_b = 2*sqrt(sigma_bQ)
7      self._a = Measurement(a, delta_a)
8      self._b = Measurement(b, delta_b)
9
10     @property
11     def info(self):
12         pm = '+' if self.b.value >= 0 else '-'
13         return f"y = a + bx: y = {self.a.value}{pm}{self.b.value}x\"
14             f"\na = {self.a}\nb = {self.b}"
15

```

Рисунок 15 — Код в файле «LinearMultipleMeasurement.py» часть 2


```

1 from math import *
2 from .LinearMultipleMeasurement import *
3
4 import matplotlib.pyplot as plt
5 from matplotlib.ticker import AutoLocator
6 import numpy as np
7
8
9 def plot_controller(method_to_decorate):
10
11     def wrapper(self, data, fig=None, ax=None, *args, **kwargs):
12         if fig is None or ax is None:
13             (fig, ax) = plt.subplots(figsize=(8, 8))
14             ax.clear()
15             method_to_decorate(self, fig, ax, data, *args, **kwargs)
16
17     return wrapper
18
19
20 class Drawer():
21     @plot_controller
22     def plot_MKN(self, fig, ax,
23                 data: LinearMultipleMeasurement,
24                 x_name='X', x_unit='', y_name='Y', y_unit='',
25                 label: str=None, pad_x=1.35, pad_y=1.65,
26                 error_X=False, error_Y=False,
27                 ):
28         nA, nB, X, Y = data.a, data.b, data.measurments_X, data.measurments_Y
29
30         if label is None:
31             label = f'График зависимости: {y_name}({x_name})'
32
33         pm = '+' if nB.value >= 0 else '-'
34         print(f'График зависимости: {y_name} = {nA.value}{pm}{nB.value}{x_name}')
35
36         # Считаем границы
37         X_min, X_max = min(X), max(X)
38         Y_min, Y_max = min(Y), max(Y)
39
40         # Экспериментальные точки с
41         xerr = [point.delta for point in X] if error_X else None
42         yerr = [point.delta for point in Y] if error_Y else None
43         ax.errorbar(x=[point.value for point in X],
44                   y=[point.value for point in Y],
45                   xerr=xerr,
46                   yerr=yerr,
47                   fmt='o', color='blue', markersize=6,
48                   label='Экспериментальные точки', capsize=3)
49
50         # Теоретическая зависимость

```

Рисунок 16 — Код в файле «Drawer.py» часть 1

```

1 X_range = np.linspace(X_min.value - X_min.delta, X_max.value + X_max.delta)
2 Y_theor = nA.value_ + X_range * nB.value_
3 ax.plot(X_range, Y_theor, 'r-', label=label)
4
5 # Устанавливаем пределы осей
6 w_x, w_y = 0, 0
7 m_x, m_y = 0.05*(X_max.value - X_min.value), 0.05*(Y_max.value - Y_min.value)
8 ax.set_xlim(
9     float(min(X_min.value - X_min.delta - m_x - w_x, min(X_range)-m_x)),
10    float(max(X_max.value + X_max.delta + m_x + w_x, max(X_range)+m_x))
11 )
12 ax.set_ylim(
13     float(min(Y_min.value - Y_min.delta - m_y - w_y, min(Y_theor)-m_y)),
14     float(max(Y_max.value + Y_max.delta + m_y + w_y, max(Y_theor)+m_y))
15 )
16 # Подписи осей
17 plt.xlabel(f'{x_name}{' if x_unit == '' else f', {x_unit}'}',
18           x=1.01, ha='right', va='bottom',
19           bbox={
20               'boxstyle': 'square', # стиль рамки (круглая/прямоугольная)
21               'facecolor': 'white', # цвет фона
22               'edgecolor': 'white', # цвет рамки
23               'pad': pad_x         # отступ текста от рамки
24           })
25 plt.ylabel(f'{y_name}{' if y_unit == '' else f', {y_unit}'}',
26           y=1.01, ha='left', va='top', rotation=0,
27           bbox={
28               'boxstyle': 'square', # стиль рамки (круглая/прямоугольная)
29               'facecolor': 'white', # цвет фона
30               'edgecolor': 'white', # цвет рамки
31               'pad': pad_y         # отступ текста от рамки
32           })
33 # Настройка делений сетки
34 ax.xaxis.set_major_locator(AutoLocator())
35 ax.yaxis.set_major_locator(AutoLocator())
36 # Включаем сетку
37 plt.grid(True, which='both', linestyle='--', alpha=0.7)
38 # Добавляем заголовок
39 plt.title(label)
40 plt.tight_layout()
41
42 """ ДЛЯ КУЧИ ГРАФИКОВ """
43
44 @plot_controller
45 async def plot_multiple_dependency(self, fig, ax,
46                                   data: List[LinearMultipleMeasurement],
47                                   x_name='X', x_unit='', y_name='Y', y_unit='',
48                                   label: str=None, labels: List[str]=None, pad_x=1.35, pad_y=1.65,
49                                   error_X=False, error_Y=False,
50                                   ):

```

Рисунок 17 — Код в файле «Drawer.py» часть 2

```

1  if label is None:
2      label = f'График зависимости: {y_name}({x_name})'
3  if labels is None:
4      labels = list(range(len(data)))
5  if len(labels) != len(data):
6      labels = ["Кол-во подписей!" for _ in range(len(data))]
7
8  # Считаем границы
9  X_min, X_max = min(lmm.min_X for lmm in data), max(lmm.max_X for lmm in data)
10 Y_min, Y_max = min(lmm.min_Y for lmm in data), max(lmm.max_Y for lmm in data)
11
12 for i in range(len(data)):
13     X = data[i].measurments_X
14     Y = data[i].measurments_Y
15     nA = data[i].a
16     nB = data[i].b
17
18     # Экспериментальные точки
19     xerr = [point.delta for point in X] if error_X else None
20     yerr = [point.delta for point in Y] if error_Y else None
21     # print(xerr, yerr)
22     ax.errorbar(x=[point.value for point in X],
23                y=[point.value for point in Y],
24                xerr=xerr,
25                yerr=yerr,
26                fmt='o', color=f'C{i}', markersize=4, capsize=1)
27
28     # Теоретическая зависимость
29     X_range = np.linspace(data[i].min_X.value - data[i].min_X.delta,
30                           data[i].max_X.value + data[i].max_X.delta)
31     Y_theor = nA.value_ + X_range * nB.value_
32     ax.plot(X_range, Y_theor, '-', label=labels[i], color=f'C{i}')
33
34 # Устанавливаем пределы осей
35 w_x, w_y = 0, 0
36 m_x, m_y = 0.05*(X_max.value - X_min.value), 0.05*(Y_max.value - Y_min.value)
37 ax.set_xlim(
38     float(min(X_min.value - X_min.delta - m_x - w_x, min(X_range)-m_x)),
39     float(max(X_max.value + X_max.delta + m_x + w_x, max(X_range)+m_x))
40 )
41 ax.set_ylim(
42     float(min(Y_min.value - Y_min.delta - m_y - w_y, min(Y_theor)-m_y)),
43     float(max(Y_max.value + Y_max.delta + m_y + w_y, max(Y_theor)+m_y))
44 )
45 # Подписи осей
46 plt.xlabel(f'{x_name}{' ' if x_unit == ' ' else f', {x_unit}'}',
47            x=1.01, ha='right', va='bottom',
48            bbox={
49                'boxstyle': 'square', # стиль рамки (круглая/прямоугольная)
50                'facecolor': 'white', # цвет фона

```

Рисунок 18 — Код в файле «Drawer.py» часть 3

```

1         'edgecolor': 'white', # цвет рамки
2         'pad': pad_x         # отступ текста от рамки
3     })
4 plt.ylabel(f'{y_name}{' if y_unit == ' ' else f', {y_unit}}',
5           y=1.01, ha='left', va='top', rotation=0,
6           bbox={
7               'boxstyle': 'square', # стиль рамки (круглая/прямоугольная)
8               'facecolor': 'white', # цвет фона
9               'edgecolor': 'white', # цвет рамки
10              'pad': pad_y         # отступ текста от рамки
11          })
12 # Настройка делений сетки
13 ax.xaxis.set_major_locator(AutoLocator())
14 ax.yaxis.set_major_locator(AutoLocator())
15 # Включаем сетку
16 plt.grid(True, which='both', linestyle='--', alpha=0.7)
17 # Добавляем заголовок
18 plt.title(label)
19 plt.legend(loc='best')
20 plt.tight_layout()

```

Рисунок 19 — Код в файле «Drawer.py» часть 4

```

1 import sympy as sp
2 from sympy.parsing.sympy_parser import parse_expr
3 from .Measurement import *
4 from .MeasException import ExpressionError
5
6 def expression(measurements: list[Measurement], s: str, name='', char='', unit=''):
7     # Создаем словарь для уникальных char (первый в приоритете)
8     measurement_dict = {}
9     for m in measurements:
10         if m.char not in measurement_dict:
11             measurement_dict[m.char] = m
12
13     # Парсим выражение и проверяем корректность
14     try:
15         expr = parse_expr(s)
16     except Exception as e:
17         raise ExpressionError(f"Некорректный ввод выражения: {str(e)}")
18
19     # Извлекаем переменные из выражения
20     variables = expr.free_symbols
21     var_names = [str(v) for v in variables]
22
23     # Проверяем наличие всех переменных в измерениях
24     for var in var_names:
25         if var not in measurement_dict:
26             raise ExpressionError(f"Неизвестная переменная: '{var}'")
27
28     # Подготавливаем данные: значения, символы, погрешности
29     symbols = {}
30     values = {}
31     deltas = {}
32     for var in variables:
33         var_str = str(var)
34         m = measurement_dict[var_str]
35         symbols[var_str] = var # Символы уже созданы при парсинге
36         values[var] = m.value_
37         # Корректируем погрешность для прямых измерений
38         deltas[var] = (2/3 * m.delta_) if m.is_direct else m.delta_
39
40     # Вычисляем среднее значение выражения
41     z_value = expr.subs(values).evalf()
42     if isinstance(z_value, sp.core.numbers.NaN):
43         raise ExpressionError("Невозможно вычислить значение выражения")
44
45     # Вычисляем абсолютную погрешность по Способу 1
46     delta_z_squared = 0
47     for var in variables:
48         derivative = sp.diff(expr, var)
49         derivative_value = derivative.subs(values).evalf()
50         delta = deltas[var]

```

Рисунок 20 — Код в файле «Expression.py» часть 1

```

1      delta_z_squared += (derivative_value * delta) ** 2
2
3      delta_z = sp.sqrt(delta_z_squared)
4      epsilon_z = (delta_z / z_value) * 100 if z_value != 0 else float('inf')
5
6      # Возвращаем результат
7      return Measurement(
8          value=float(z_value),
9          delta=float(delta_z),
10         epsilon=float(epsilon_z),
11         name=name,
12         char=char,
13         unit=unit
14     )

```

Рисунок 21 — Код в файле «Expression.py» часть 2

```

1 class MeasException(BaseException):
2     pass
3
4 class StudentException(BaseException):
5     def __init__(self, N: int):
6         self.N = N
7         super().__init__(f"Отсутствует коэффициент Стьюдента N = {N}!")
8
9 class ExpressionError(BaseException):
10     pass

```

Рисунок 22 — Код в файле «MeasException.py» часть 1

```

1 from .Tools import *
2 from .Measurement import *
3 from .DirectMultipleMeasurement import *
4 from .MultipleMeasurement import *
5 from .LinearMultipleMeasurement import *
6 from .Drawer import *
7 from .MeasException import *
8 from .Expression import *

```

Рисунок 23 — Код в файле «__init__.py» часть 1

```

1 from tkinter import ttk, simpledialog, messagebox
2 from EntryEx import EntryEx
3
4
5 class AskNameDialog(simpledialog.Dialog):
6     """ Расширенный диалог ввода string """
7     result: str
8
9     def body(self, master):
10         ttk.Label(master, text="Введите название эксперимента:").pack()
11         self.el = EntryEx(master)
12         self.el.pack(expand=True, fill="both")
13         return self.el
14
15     def apply(self):
16         self.result = self.el.get()
17         if len(self.result) > 100:
18             messagebox.showwarning("Ошибка", "Слишком длинный ввод!")
19         if self.result.replace(' ', '') == '':
20             messagebox.showwarning("Ошибка", "Пустой ввод!")

```

Рисунок 24 — Код в файле «AskNameDialog.py» часть 1

```

1 from tkinter import *
2 from tkinter import ttk
3 from EntryEx import EntryEx
4 from SaveMeasFrame import SaveMeasFrame
5 from meas import DirectMultipleMeasurement, MeasException, StudentException
6 from typing import Callable
7 import re
8
9 float_re = re.compile(r'^[+-]?(\d+([.]\d*))?([eE][+-]?\d+)?|\.\d+([eE][+-]?\d+)?$')
10
11
12 class DMMForm(ttk.Frame):
13     dmm: DirectMultipleMeasurement|None
14
15     def __init__(self, master: Misc|None, apply_command: Callable, *args, **kwargs):
16         super().__init__(master, *args, **kwargs)
17         self.create_measurement = apply_command
18         self.dmm = None
19         self._create_widgets()
20         self.update_info()
21
22     def _create_widgets(self):
23         # Левая панель
24         self.left_frame = ttk.Frame(self, padding=10)
25         self.left_frame.pack(side=LEFT, fill=BOTH, expand=True)
26
27         # Текстовое поле с прокруткой
28         self.text = Text(self.left_frame, wrap=None, width=20)
29         scroll = ttk.Scrollbar(self.left_frame, command=self.text.yview)
30         self.text.configure(yscrollcommand=scroll.set)
31         self.text.pack(side=LEFT, fill=BOTH, expand=True)
32         scroll.pack(side=RIGHT, fill=Y)
33
34         # Правая панель
35         self.right_frame = ttk.Frame(self, padding=10)
36         self.right_frame.pack(side=LEFT, fill=BOTH, expand=True)
37
38         # Инструментальная погрешность
39         self.delta_form = ttk.LabelFrame(self.right_frame,
40                                         text="Инструментальная погрешность")
41         self.delta_var = StringVar(self.delta_form, value="")
42         self.delta_entry = EntryEx(self.delta_form, textvariable=self.delta_var)
43         self.delta_entry.configure(
44             validate="key",
45             validatecommand=(self.register(self._validate_float), '%P')
46         )
47         self.delta_entry.pack(fill=X, expand=True, padx=5, pady=5)
48         self.delta_form.pack(fill=BOTH, expand=True, padx=5, pady=5)
49
50         # Вывод результата

```

Рисунок 25 — Код в файле «DMMForm.py» часть 1


```

1 self.result_form = ttk.LabelFrame(self.right_frame, text="Результат")
2 self.result_var = StringVar(self.result_form, value="Введите значения!")
3 self.result_label = ttk.Label(self.result_form, textvariable=self.result_var)
4 self.result_label.pack(fill=BOTH, expand=True, padx=5, pady=5)
5 self.result_form.pack(fill=BOTH, expand=True, padx=5, pady=5)
6
7 # Сохранение
8 self.meas_form = SaveMeasFrame(self.right frame, self. create_meas,
9                               text="Сохранение результата")
10 self.meas_form.pack(fill=BOTH, expand=True, padx=5, pady=5)
11
12 # Привязка валидации
13 self.text.bind("<<Modified>>", self._validate_input)
14
15 def _validate_input(self, event):
16     """ Валидация списка значений """
17     if not self.text.edit_modified():
18         return
19
20     position = self.text.index(INSERT)
21     text = self.text.get("1.0", END)
22     lines = text.splitlines()
23     valid_lines = []
24
25     for line in lines:
26         if line in ["-", ""]:
27             valid_lines.append(line.strip())
28             continue
29         try:
30             float(line)
31             valid_lines.append(line)
32         except ValueError:
33             continue
34
35     self.text.delete("1.0", END)
36     self.text.insert("1.0", "\n".join(valid_lines))
37     self.text.edit_modified(False)
38     self.text.mark_set("insert", position)
39
40     self.update_info()
41
42 def update_info(self):
43     """ Перерасчёт результата измерения """
44     text = self.text.get("1.0", END)
45     values = [
46         float(line) for line in text.splitlines() if line not in ["-", ""]
47     ]
48     try:
49         try:
50             delta_ins = float(self.delta_var.get())

```

Рисунок 26 — Код в файле «DMMForm.py» часть 2

```

1         except ValueError:
2             delta_ins = 0
3             self.dmm = DirectMultipleMeasurement(values, delta_ins)
4             self.result_var.set(self.dmm.rounded)
5         except StudentException as e:
6             self.result_var.set(e)
7             self.dmm = None
8         except MeasException as e:
9             self.dmm = None
10            self.result_var.set("Введите значения!")
11
12     def _validate_float(self, value: str):
13         """ Валидация ввода для float значений """
14         return float_re.match(value) is not None or value in ["", "-"]
15
16     def _create_meas(self, name: str, char: str, unit: str):
17         """ Привязка имени, символа и единицы измерения к результату """
18         if self.dmm is None:
19             return
20         self.dmm.name = name
21         self.dmm.char = char
22         self.dmm.unit = unit
23         self.create_measurement(None, self.dmm.asMeasurment)

```

Рисунок 27 — Код в файле «DMMForm.py» часть 3

```

1 from tkinter import ttk, Menu
2
3
4 class EntryEx(ttk.Entry):
5     """ Расширенный entry с функциями cut/copy/paste """
6
7     def __init__(self, *args, **kwargs):
8         super().__init__(*args, **kwargs)
9         self.menu = Menu(self, tearoff=False)
10        self.menu.add_command(label="Copy", command=self.popup_copy)
11        self.menu.add_command(label="Cut", command=self.popup_cut)
12        self.menu.add_command(label="Paste", command=self.popup_paste)
13        self.bind("<Button-3>", self.display_popup)
14
15    def display_popup(self, event):
16        self.menu.post(event.x_root, event.y_root)
17
18    def popup_copy(self):
19        self.event_generate("<<Copy>>")
20
21    def popup_cut(self):
22        self.event_generate("<<Cut>>")
23
24    def popup_paste(self):
25        self.event_generate("<<Paste>>")

```

Рисунок 28 — Код в файле «EntryEx.py» часть 1

```

1 from tkinter import *
2 from tkinter import ttk, messagebox
3 from EntryEx import *
4 from meas import Measurement, expression, ExpressionError
5 from typing import Callable
6
7
8 class ExpressionForm(ttk.Frame):
9     measurments_list: list[Measurement]
10
11     def __init__(self, master: Misc | None, apply_command: Callable,
12                 measurments_list: list[Measurement], *args, **kwargs):
13         super().__init__(master, *args, **kwargs)
14         self._create_widgets()
15         self.measurments_list = measurments_list
16         self.apply_command = apply_command
17
18     def _create_widgets(self):
19         # Подпись
20         self.name_label = ttk.Label(self, text="Ввод выражения")
21         self.name_label.grid(row=1, column=0, padx=5, pady=5, sticky="")
22
23         # Name (обязательное, до 80 символов)
24         self.name_label = ttk.Label(self, text="Название*")
25         self.name_label.grid(row=2, column=0, padx=5, pady=5, sticky="e")
26         self.name_entry = EntryEx(self)
27         self.name_entry.grid(row=2, column=1, columnspan=3,
28                             padx=5, pady=5, sticky="ew")
29         self.name_entry.configure(
30             validate="key",
31             validatecommand=(self.register(lambda text: len(text) <= 80), '%P')
32         )
33
34         # Char (до 40 символов)
35         self.char_label = ttk.Label(self, text="Символ")
36         self.char_label.grid(row=3, column=0, padx=5, pady=5, sticky="e")
37         self.char_entry = EntryEx(self)
38         self.char_entry.grid(row=3, column=1, padx=5, pady=5, sticky="ew")
39         self.char_entry.configure(
40             validate="key",
41             validatecommand=(self.register(lambda text: len(text) <= 40), '%P')
42         )
43
44         # Unit (до 40 символов)
45         self.unit_label = ttk.Label(self, text="Ед. изм.")
46         self.unit_label.grid(row=3, column=2, padx=5, pady=5, sticky="e")
47         self.unit_entry = EntryEx(self)
48         self.unit_entry.grid(row=3, column=3, padx=5, pady=5, sticky="ew")
49         self.unit_entry.configure(
50             validate="key",

```

Рисунок 29 — Код в файле «ExpressionForm.py» часть 1

```

1         validatecommand=(self.register(lambda text: len(text) <= 40), '%P')
2     )
3
4     # Expression (обязательное)
5     self.expression_label = ttk.Label(self, text="Выражение")
6     self.expression_label.grid(row=4, column=0, padx=5, pady=5, sticky="e")
7     self.expression_entry = EntryEx(self)
8     self.expression_entry.grid(row=4, column=1, columnspan=3,
9                                padx=5, pady=5, sticky="ew")
10
11     # Кнопки
12     self.apply_button = ttk.Button(self, text="Рассчитать",
13                                    command=self.apply)
14     self.apply_button.grid(row=5, column=1, padx=5, pady=5, sticky="ew")
15
16     self.clear_button = ttk.Button(self, text="Отмена",
17                                    command=self.clear_form)
18     self.clear_button.grid(row=5, column=2, padx=5, pady=5, sticky="ew")
19
20     # Расширение кнопок
21     self.grid_columnconfigure(1, weight=1)
22     self.grid_columnconfigure(3, weight=1)
23
24     def clear_form(self):
25         """ Очистка всех полей ввода """
26         self.name_entry.delete(0, END)
27         self.unit_entry.delete(0, END)
28         self.char_entry.delete(0, END)
29         self.expression_entry.delete(0, END)
30
31     def apply(self):
32         """ Обработка нажатия кнопки сохранения """
33         try:
34             name = self.name_entry.get()
35             if name.replace(' ', '') == '':
36                 raise ExpressionError("Имя обязательно!")
37
38             char = self.char_entry.get()
39             unit = self.unit_entry.get()
40
41             expr = self.expression_entry.get()
42             if expr.replace(' ', '') == '':
43                 raise ExpressionError("Выражение обязательно!")
44
45             ret = expression(self.measurements_list, expr, name, char, unit)
46
47             self.apply_command(None, ret)
48
49         except ExpressionError as e:
50             messagebox.showwarning("Некорректное выражение", str(e))

```

Рисунок 30 — Код в файле «ExpressionForm.py» часть 2

```
1     except Exception as e:  
2         messagebox.showerror("Неожиданная ошибка!", str(e))  
3         print(e)
```

Рисунок 31 — Код в файле «ExpressionForm.py» часть 3

```

1 from tkinter import *
2 from tkinter import ttk
3 from EntryEx import EntryEx
4 from typing import Callable
5
6
7 class GraphicSettingsFrame(ttk.LabelFrame):
8     def __init__(self, master: Misc|None, apply_command: Callable,
9                 save_command: Callable, *args, **kwargs):
10         super().__init__(master, *args, **kwargs)
11         self.apply_command = apply_command
12         self.save_command = save_command
13         self._create_widgets()
14
15     def _create_widgets(self):
16         # Инициализация переменных
17         self.x_name_var = StringVar(value="X")
18         self.y_name_var = StringVar(value="Y")
19         self.x_unit_var = StringVar(value="")
20         self.y_unit_var = StringVar(value="")
21         self.pad_x_var = DoubleVar(value=1.35)
22         self.pad_y_var = DoubleVar(value=1.65)
23         self.error_x_var = BooleanVar(value=False)
24         self.error_y_var = BooleanVar(value=False)
25
26         # Строка 0: Названия осей
27         self.x_name_label = ttk.Label(self, text="Название X:")
28         self.x_name_label.grid(row=0, column=0, padx=2, pady=2, sticky=W)
29
30         self.x_name_entry = EntryEx(self, textvariable=self.x_name_var, width=15)
31         self.x_name_entry.grid(row=0, column=1, padx=2, pady=2)
32
33         self.y_name_label = ttk.Label(self, text="Название Y:")
34         self.y_name_label.grid(row=0, column=2, padx=2, pady=2, sticky=W)
35
36         self.y_name_entry = EntryEx(self, textvariable=self.y_name_var, width=15)
37         self.y_name_entry.grid(row=0, column=3, padx=2, pady=2)
38
39         # Строка 1: Единицы измерения
40         self.x_unit_label = ttk.Label(self, text="Единицы X:")
41         self.x_unit_label.grid(row=1, column=0, padx=2, pady=2, sticky=W)
42
43         self.x_unit_entry = EntryEx(self, textvariable=self.x_unit_var, width=15)
44         self.x_unit_entry.grid(row=1, column=1, padx=2, pady=2)
45
46         self.y_unit_label = ttk.Label(self, text="Единицы Y:")
47         self.y_unit_label.grid(row=1, column=2, padx=2, pady=2, sticky=W)
48
49         self.y_unit_entry = EntryEx(self, textvariable=self.y_unit_var, width=15)
50         self.y_unit_entry.grid(row=1, column=3, padx=2, pady=2)

```

Рисунок 32 — Код в файле «GraphicSettingsFrame.py» часть 1


```

1  # Строка 2: Отступы
2  self.pad_x_label = ttk.Label(self, text="Отступ X:")
3  self.pad_x_label.grid(row=2, column=0, padx=2, pady=2, sticky=W)
4
5  self.pad_x_slider = ttk.Scale(
6      self, from_=0.9, to=2.0, variable=self.pad_x_var,
7      orient=HORIZONTAL, length=120
8  )
9  self.pad_x_slider.grid(row=2, column=1, padx=2, pady=2)
10
11 self.pad_y_label = ttk.Label(self, text="Отступ Y:")
12 self.pad_y_label.grid(row=2, column=2, padx=2, pady=2, sticky=W)
13
14 self.pad_y_slider = ttk.Scale(
15     self, from_=0.9, to=2.0, variable=self.pad_y_var,
16     orient=HORIZONTAL, length=120
17 )
18 self.pad_y_slider.grid(row=2, column=3, padx=2, pady=2)
19
20 # Строка 3: Чекбоксы
21 self.error_x_check = ttk.Checkbutton(
22     self,
23     text="Ошибки X",
24     variable=self.error_x_var
25 )
26 self.error_x_check.grid(row=3, column=0, columnspan=2,
27     padx=2, pady=2, sticky=W)
28
29 self.error_y_check = ttk.Checkbutton(
30     self,
31     text="Ошибки Y",
32     variable=self.error_y_var
33 )
34 self.error_y_check.grid(row=3, column=2, columnspan=2,
35     padx=2, pady=2, sticky=W)
36
37 # Строка 4: Кнопки
38 self.apply_button = ttk.Button(self, text="Применить",
39     command=self.apply_command)
40 self.apply_button.grid(row=4, column=1, padx=5, pady=5, sticky="ew")
41
42 self.clear_button = ttk.Button(self, text="Сбросить",
43     command=self.clear_form)
44 self.clear_button.grid(row=4, column=2, padx=5, pady=5, sticky="ew")
45
46 self.save_button = ttk.Button(self, text="Сохранить график",
47     command=self.save_command)
48 self.save_button.grid(row=4, column=3, padx=5, pady=5, sticky="ew")
49

```

Рисунок 33 — Код в файле «GraphicSettingsFrame.py» часть 2


```
1      # Настройка веса колонок для растяжения
2      for i in range(4):
3          self.columnconfigure(i, weight=1)
4
5      def clear_form(self):
6          """ Очистка/сброс всех полей ввода """
7          self.x_name_var.set("X")
8          self.y_name_var.set("Y")
9          self.x_unit_var.set("")
10         self.y_unit_var.set("")
11         self.pad_x_var.set(1.35)
12         self.pad_y_var.set(1.65)
13         self.error_x_var.set(False)
14         self.error_y_var.set(False)
```

Рисунок 34 — Код в файле «GraphicSettingsFrame.py» часть 3

```

1 from tkinter import *
2 from tkinter import ttk
3
4 from EntryEx import *
5 from MeasForm import *
6
7
8 class ListNotebook(Frame):
9     """ Отображает коллекцию и формы взаимодействия с ними """
10     _frames: list[ttk.Frame]
11     _items: list[Measurement]
12     _shift: int
13     _current_tab: int|None
14
15     @property
16     def items(self):
17         return self._items
18
19     def __init__(self):
20         super().__init__()
21         self._frames = []
22         self._items = []
23         self._shift = 0
24
25         # NOTEBOOK
26         self.notebook = ttk.Notebook(self)
27         ttk.Style(self).layout("TNotebook.Tab", [])
28         self.notebook.pack(expand=True, side="right", fill="both")
29
30         # LISTBOX
31         self.left_frame = ttk.Frame(self, width=180)
32         self.left_frame.pack_propagate(False)
33         self.left_frame.pack(side="left", fill="y", expand=False)
34         self.listbox = Listbox(self.left_frame)
35
36         # Scrollbar для Listbox
37         scrollbar = ttk.Scrollbar(self.listbox, orient="vertical",
38                                   command=self.listbox.yview)
39         scrollbar.pack(side=RIGHT, fill=Y)
40         self.listbox["yscrollcommand"] = scrollbar.set
41
42     def selected(event):
43         """ Переключение вкладок """
44         if len(self.listbox.curselection()) != 0:
45             tab = self.listbox.curselection()[0]
46             self._current_tab = tab
47             self.notebook.select(self._shift + tab)
48
49     self.listbox.bind("<<ListboxSelect>>", selected)
50     self.listbox.pack(expand=True, fill="both")

```

Рисунок 35 — Код в файле «ListNotebook.py» часть 1

```

1  def basic_frame(self, frame: ttk.Frame):
2      """ Добавляет основные формы, не привязанные к измерениям """
3      self._shift += 1
4      self.notebook.add(frame)
5
6  def open_frame(self, frame: ttk.Frame):
7      """ Открывает форму программно """
8      self.notebook.select(frame)
9
10 def add(self, frame: MeasForm):
11     """ Добавляет измерения и их формы """
12     element = frame.current_meas
13     self._frames.append(frame)
14     self._items.append(element)
15     self.listbox.insert(END, element.name)
16     self.notebook.add(frame)
17
18 def update_listbox(self, element: Measurement):
19     """ Обновляет пункты меню listbox """
20     index = self._current_tab
21     self._items[index] = element
22     self.listbox.delete(index)
23     self.listbox.insert(index, self._items[index].name)
24
25 def delete(self, index: int):
26     """ Удаляет пункт списка измерений по индексу """
27     self.listbox.delete(index)
28     self.notebook.forget(self._frames[index])
29     self._frames.pop(index)
30     self._items.pop(index)
31
32 def pop(self):
33     """ Удаляет текущий выбранный пункт списка измерений """
34     if self.listbox.curselection():
35         tab = self.listbox.curselection()[0]
36         ret = self._items[tab]
37         self.delete(tab)
38         return ret
39     return None
40
41 def clear(self):
42     """ Очистка списка измерений """
43     for i in range(len(self._items) - 1, -1, -1):
44         self.delete(i)

```

Рисунок 36 — Код в файле «ListNotebook.py» часть 2

```

1 from tkinter import *
2 from tkinter import ttk, filedialog, messagebox
3 from GraphicSettingsFrame import GraphicSettingsFrame
4 from SaveMeasFrame import SaveMeasFrame
5 import matplotlib.pyplot as plt
6 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
7 from meas import LinearMultipleMeasurement, MeasException, Drawer
8 from typing import Callable
9
10
11 class LMMForm(ttk.Frame):
12     lmm: LinearMultipleMeasurement|None
13
14     def __init__(self, master: Misc|None,
15                 apply_command: Callable, *args, **kwargs):
16         super().__init__(master, *args, **kwargs)
17         self.create_measurement = apply_command
18         self.lmm = None
19         self._create_widgets()
20         self.update_info()
21
22     def _create_widgets(self):
23         # Левая панель
24         self.left_frame = ttk.Frame(self, padding=10)
25         self.left_frame.pack(side=LEFT, fill=BOTH, expand=True)
26
27         # Текстовое поле с прокруткой
28         self.text = Text(self.left_frame, wrap=NONE, width=20)
29         scroll = ttk.Scrollbar(self.left_frame, command=self.text.yview)
30         self.text.configure(yscrollcommand=scroll.set)
31
32         self.text.pack(side=LEFT, fill=BOTH, expand=True)
33         scroll.pack(side=RIGHT, fill=Y)
34
35         # Привязка валидации
36         self.text.bind("<<Modified>>", self._validate_input)
37
38         # Правая панель
39         self.right_frame = ttk.Frame(self, padding=10)
40         self.right_frame.pack(side=RIGHT, fill=BOTH, expand=True)
41
42         # Информационная панель
43         info_frame = ttk.Frame(self.right_frame)
44         info_frame.pack(side=TOP, fill=X)
45
46         self.count_label = ttk.Label(info_frame,
47                                     text="Количество значений: 0")
48         self.a_label = ttk.Label(info_frame, text=f"A")
49         self.b_label = ttk.Label(info_frame, text=f"B")
50         self.eq_label = ttk.Label(info_frame,

```

Рисунок 37 — Код в файле «LMMForm.py» часть 1

```

1         text="Уравнение:  $y = Ax + B$ ")
2
3     self.count_label.pack(anchor=W)
4     self.a_label.pack(anchor=W)
5     self.b_label.pack(anchor=W)
6     self.eq_label.pack(anchor=W)
7
8     # График
9     self.figure, self.ax = plt.subplots(figsize=(5, 4))
10    self.canvas = FigureCanvasTkAgg(self.figure, self.right_frame)
11    self.canvas.get_tk_widget().pack(side=TOP, fill=BOTH, expand=True)
12
13    # Центральная панель
14    self.center_frame = ttk.Frame(self)
15    self.center_frame.pack(fill=BOTH, expand=True, padx=5, pady=5)
16
17    # Фрейм конфигурации
18    self.config_frame = GraphicSettingsFrame(
19        self.center_frame, self.update_graphic,
20        self.save_plot, text="Настройка графика"
21    )
22    self.config_frame.pack(fill=BOTH, expand=True, pady=5)
23
24    # Формы вывода коэффициентов
25    self.a_frame = SaveMeasFrame(self.center_frame,
26                                self.create_meas_a,
27                                text="Свободный член")
28    self.a_frame.pack(fill=BOTH, expand=True, pady=5)
29
30    self.b_frame = SaveMeasFrame(self.center_frame,
31                                self.create_meas_b,
32                                text="Угловой коэффициент")
33    self.b_frame.pack(fill=BOTH, expand=True, pady=5)
34
35    def _validate_input(self, event):
36        """ Валидация списка значений """
37        if not self.text.edit_modified():
38            return
39
40        position = self.text.index(INSERT)
41        text = self.text.get("1.0", END)
42        lines = text.splitlines()
43        valid_lines = []
44
45        for line in lines:
46            lsp = line.split()
47            if len(lsp) < 2:
48                valid_lines.append(line)
49                continue
50            if len(lsp) > 2:

```

Рисунок 38 — Код в файле «LMMForm.py» часть 2

```

1         lsp = lsp[0:2]
2     if lsp[1].strip() == "-":
3         valid_lines.append(f'{lsp[0]} {lsp[1]}')
4         continue
5     try:
6         float(lsp[0].strip())
7         float(lsp[1].strip())
8         valid_lines.append(line)
9     except ValueError:
10        continue
11
12    self.text.delete("1.0", END)
13    self.text.insert("1.0", "\n".join(valid_lines))
14    self.text.edit_modified(False)
15    self.text.mark_set("insert", position)
16
17    self.update_info()
18
19    def update_info(self):
20        """ Перерасчёт результата """
21        text = self.text.get("1.0", END)
22        values = []
23        for line in text.splitlines():
24            if len(line.split()) == 2 and line.split()[1] != '-':
25                values.append(line.strip().split())
26
27        try:
28            # Расчёт
29            x = [float(v[0]) for v in values]
30            y = [float(v[1]) for v in values]
31            self.lmm = LinearMultipleMeasurement(x, y)
32
33            # Обновление информации
34            self.count_label.config(
35                text=f"Количество значений: {self.lmm._N}")
36            self.a_label.config(
37                text=f"Угловой коэффициент: {self.lmm.b.rounded}")
38            self.b_label.config(
39                text=f"Свободный член: {self.lmm.a.rounded}")
40            self.eq_label.config(
41                text=f"Уравнение: y = {self.lmm.b.value}x + {self.lmm.a.value}")
42
43            self._update_graphic()
44
45        except MeasException as e:
46            self.count_label.config(text=e)
47            self.a_label.config(text=f"Угловой коэффициент K")
48            self.b_label.config(text=f"Свободный член A")
49            self.eq_label.config(text=f"Уравнение: y = Kx + A")
50            self.lmm = None

```

Рисунок 39 — Код в файле «LMMForm.py» часть 3

```

1         self.ax.clear()
2
3     def _update_graphic(self):
4         """ Обновление графика """
5         if self.lmm is None:
6             return
7
8         Drawer().plot_MKN(
9             fig=self.figure,
10            ax=self.ax,
11            data=self.lmm,
12            x_name=self.config_frame.x_name_var.get(),
13            x_unit=self.config_frame.x_unit_var.get(),
14            y_name=self.config_frame.y_name_var.get(),
15            y_unit=self.config_frame.y_unit_var.get(),
16            pad_x=self.config_frame.pad_x_var.get(),
17            pad_y=self.config_frame.pad_y_var.get(),
18            error_X=self.config_frame.error_x_var.get(),
19            error_Y=self.config_frame.error_y_var.get()
20        )
21        self.canvas.draw()
22
23    def _create_meas_a(self, name: str, char: str, unit: str):
24        """ Сохранение измерения А """
25        if self.lmm is None:
26            return
27        self.lmm.a.name = name
28        self.lmm.a.char = char
29        self.lmm.a.unit = unit
30        self.create_measurement(None, self.lmm.a)
31
32    def _create_meas_b(self, name: str, char: str, unit: str):
33        """ Сохранение измерения В """
34        if self.lmm is None:
35            return
36        self.lmm.b.name = name
37        self.lmm.b.char = char
38        self.lmm.b.unit = unit
39        self.create_measurement(None, self.lmm.b)
40
41    def save_plot(self):
42        """ Сохранение графика """
43        try:
44            path = filedialog.asksaveasfilename(
45                initialdir = "/",
46                title = "Сохранение",
47                filetypes = (("jpeg files", "*.jpg"), ("all files", "*.*"))
48            )
49            self.figure.savefig(path)
50        except PermissionError as e:

```

Рисунок 40 — Код в файле «LMMForm.py» часть 4

```
1     messagebox.showerror("Ошибка",  
2                           "Недостаточно прав для сохранения!")  
3     print(e)  
4 except Exception as e:  
5     messagebox.showerror("Неожиданная ошибка", str(e))  
6     print(e)
```

Рисунок 41 — Код в файле «LMMForm.py» часть 5


```

1 from tkinter import *
2 from tkinter import ttk, messagebox
3
4 from AskNameDialog import AskNameDialog
5 from ListNotebook import ListNotebook
6 from MeasForm import MeasForm
7 from ExpressionForm import ExpressionForm
8 from LMMForm import LMMForm
9 from DMMForm import DMMForm
10
11 from Models import *
12 from meas import *
13
14
15 class App(Tk):
16     current_exp: Experiment|None
17
18     def __init__(self):
19         super().__init__()
20         self.current_exp = None
21         self.title("Выберите эксперимент...")
22         self.minsize(1300, 600)
23         self.create_menu()
24         self.create_widgets()
25
26         self.load_experiments()
27
28     def create_widgets(self):
29         # Панель действий
30         self.bottom_frame = Frame(self, height=50)
31         self.bottom_frame.pack_propagate(False)
32         self.bottom_frame.pack(side="bottom", fill="x", expand=False)
33
34         # ListNotebook
35         self.note = ListNotebook()
36         self.note.pack(expand=True, side="top", fill="both")
37
38         # Пустая форма
39         self.empty_frame = ttk.Frame(self)
40         ttk.Button(self.empty_frame, text="Новый эксперимент",
41                   command=self.new_experiment).pack(expand=True)
42         self.note.basic_frame(self.empty_frame)
43
44         # Форма добавления Measurement
45         self.add_frame = MeasForm(self, self.create_measurement)
46         c = lambda frame=self.add_frame: self.note.open_frame(frame)
47         ttk.Button(self.bottom_frame, text="Добавить",
48                   command=c).grid(row=0, column=0)
49         self.edit_menu.add_cascade(label="Добавить", command=c)
50         self.note.basic_frame(self.add_frame)

```

Рисунок 42 — Код в файле «main.py» часть 1

```

1
2 # Форма вычисления выражения
3 self.expr_form = ExpressionForm(self, self.create_measurement,
4                                 self.note._items)
5 c = lambda frame=self.expr_form: self.note.open_frame(frame)
6 ttk.Button(self.bottom_frame, text="Выражение",
7            command=c).grid(row=0, column=1)
8 self.edit_menu.add_cascade(label="Выражение", command=c)
9 self.note.basic_frame(self.expr_form)
10
11 # Форма вычисления LMM
12 self.lmm_form = LMMForm(self, self.create_measurement)
13 c = lambda frame=self.lmm_form: self.note.open_frame(frame)
14 ttk.Button(self.bottom_frame, text="LMM",
15            command=c).grid(row=0, column=2)
16 self.edit_menu.add_cascade(label="LMM", command=c)
17 self.note.basic_frame(self.lmm_form)
18
19 # Форма вычисления DMM
20 self.dmm_form = DMMForm(self, self.create_measurement)
21 c = lambda frame=self.dmm_form: self.note.open_frame(frame)
22 ttk.Button(self.bottom_frame, text="DMM",
23            command=c).grid(row=0, column=3)
24 self.edit_menu.add_cascade(label="DMM", command=c)
25 self.note.basic_frame(self.dmm_form)
26
27 # Кнопка удаления
28 ttk.Button(self.bottom_frame, text="Удалить",
29            command=self.delete_measurement).grid(row=0, column=4)
30 self.edit_menu.add_cascade(
31     label="Удалить",
32     command=self.delete_measurement
33 )
34
35 def create_menu(self):
36     self.option_add("*tearOff", FALSE)
37
38     self.exp_menu = Menu()
39     self.edit_menu = Menu()
40
41     self.file_menu = Menu()
42     self.file_menu.add_cascade(label="База данных...",
43                               command=self.choose_bd)
44     self.file_menu.add_separator()
45     self.file_menu.add_cascade(label="Новый эксперимент",
46                               command=self.new_experiment)
47     self.file_menu.add_cascade(label="Удалить эксперимент",
48                               command=self.delete_experiment)
49     self.file_menu.add_cascade(label="Эксперимент",
50                               menu=self.exp_menu)

```

Рисунок 43 — Код в файле «main.py» часть 2

```

1      menu = Menu()
2      menu.add_cascade(label="Файл", menu=self.file_menu)
3      menu.add_cascade(label="Правка", menu=self.edit_menu)
4      menu.add_cascade(label="Справка", command=self.reference)
5      self.config(menu=menu)
6
7      """ Информационные сообщения """
8
9      def choose_bd(self):
10         messagebox.showinfo(
11             title="База данных",
12             message=f"Используемая база данных: {PATH}"
13         )
14
15         def reference(self):
16             messagebox.showinfo(
17                 title="Справка",
18                 message="Разработчик: Сакулин Иван Михайлович (467335)"
19             )
20
21         """ Управление экспериментами """
22
23         def new_experiment(self):
24             result = AskNameDialog(self, "Новый эксперимент").result
25             if result is not None:
26                 exp = Experiment.create(name=result)
27                 self.load_experiments()
28                 self.open_experiment(exp)
29
30         def delete_experiment(self):
31             if self.current_exp is not None:
32                 result = messagebox.askokcancel(
33                     "Удаление",
34                     "Удалить эксперимент?"
35                 )
36                 if result:
37                     Experiment.delete_by_id(self.current_exp.exp_id)
38                     self.note.clear()
39                     self.title("Выберите эксперимент...")
40                     self.note.notebook.select(self.empty_frame)
41                     self.load_experiments()
42
43         def load_experiments(self):
44             self.exp_menu.delete(0, END)
45             for exp in Experiment.select():
46                 self.exp_menu.add_radiobutton(
47                     label=exp.name,
48                     command=lambda exp=exp: self.open_experiment(exp))
49             if Experiment.select():

```

Рисунок 44 — Код в файле «main.py» часть 3

```

1         self.file_menu.entryconfigure(3, state=NORMAL)
2         self.file_menu.entryconfigure(4, state=NORMAL)
3         self.open_experiment(exp)
4     else:
5         self.file_menu.entryconfigure(3, state=DISABLED)
6         self.file_menu.entryconfigure(4, state=DISABLED)
7
8     def open_experiment(self, exp):
9         self.note.clear()
10        self.current_exp = exp
11        self.title(exp.name)
12        for m in Meas.select().where(
13            (Meas.exp_id == exp.exp_id) & (Meas.name.is_null(False))):
14            ms = Measurement(m.value, m.delta, m.epsilon, m.name,
15                           m.char, m.unit, direct=m.direct)
16            ms.mid = m.meas_id
17            self.load_measurement(ms)
18        self.note.notebook.select(self.add_frame)
19
20    """ Управление списком измерений """
21
22    def create_measurement(self, _, m: Measurement):
23        m.mid = Meas.create(
24            exp_id=self.current_exp,
25            name=m.name,
26            char=m.char,
27            unit=m.unit,
28            value=m.value_,
29            delta=m.delta_,
30            epsilon=m.epsilon_,
31            direct=m.is_direct
32        ).meas_id
33        self.load_measurement(m)
34
35    def edit_measurement(self, old: Measurement, m: Measurement):
36        meas = Meas.get(Meas.meas_id == old.mid)
37        meas.name=m.name
38        meas.char=m.char
39        meas.unit=m.unit
40        meas.value=m.value_
41        meas.delta=m.delta_
42        meas.epsilon=m.epsilon_
43        meas.direct=m.is_direct
44        meas.save()
45        self.note.update_listbox(m)
46
47    def load_measurement(self, m: Measurement):
48        frame = MeasForm(self, self.edit_measurement, m)
49        self.note.add(frame)
50

```

Рисунок 45 — Код в файле «main.py» часть 4

```
1      def delete_measurement(self):
2          ret = self.note.pop()
3          if ret is None:
4              return
5          Meas.delete_by_id(ret.mid)
6
7  if __name__ == "__main__":
8      PATH = "test.db"
9      db.init(PATH)
10     db.create_tables(db_tables)
11     root = App()
12     root.mainloop()
13
```

Рисунок 46 — Код в файле «main.py» часть 5

```

1 from tkinter import *
2 from tkinter import ttk, messagebox
3 from EntryEx import EntryEx
4 from meas import Measurement
5 from typing import Callable
6 import re
7
8 float_re = re.compile(r'^[+-]?(\d+([.]\d*)?([eE][+-]?\d+)?|\.\d+([eE][+-]?\d+)?)$')
9
10
11 class MeasForm(ttk.Frame):
12     def __init__(self, master: Misc|None, apply_command: Callable,
13                 meas: Measurement|None=None, *args, **kwargs):
14         super().__init__(master, *args, **kwargs)
15         self.apply_command = apply_command
16         self._create_widgets()
17
18         self.current_meas = meas
19         if meas is not None:
20             self._fill(meas)
21         if meas is None:
22             self.clear_button.config(text="Очистить")
23
24     def _create_widgets(self):
25         # Подпись
26         self.name_label = ttk.Label(self, text="Простое измерение")
27         self.name_label.grid(row=1, column=0, padx=5, pady=5, sticky="")
28
29         # Name (до 80 символов)
30         self.name_label = ttk.Label(self, text="Название*")
31         self.name_label.grid(row=2, column=0, padx=5, pady=5, sticky="e")
32         self.name_entry = EntryEx(self)
33         self.name_entry.grid(row=2, column=1, columnspan=3,
34                             padx=5, pady=5, sticky="ew")
35         self.name_entry.configure(
36             validate="key",
37             validatecommand=(self.register(lambda text: len(text) <= 80), '%P')
38         )
39
40         # Char (до 40 символов)
41         self.char_label = ttk.Label(self, text="Символ")
42         self.char_label.grid(row=3, column=0, padx=5, pady=5, sticky="e")
43         self.char_entry = EntryEx(self)
44         self.char_entry.grid(row=3, column=1, padx=5, pady=5, sticky="ew")
45         self.char_entry.configure(
46             validate="key",
47             validatecommand=(self.register(lambda text: len(text) <= 40), '%P')
48         )
49
50         # Unit (до 40 символов)

```

Рисунок 47 — Код в файле «MeasForm.py» часть 1


```

1 self.unit_label = ttk.Label(self, text="Ед. изм.")
2 self.unit_label.grid(row=3, column=2, padx=5, pady=5, sticky="e")
3 self.unit_entry = EntryEx(self)
4 self.unit_entry.grid(row=3, column=3, padx=5, pady=5, sticky="ew")
5 self.unit_entry.configure(
6     validate="key",
7     validatecommand=(self.register(lambda text: len(text) <= 40), '%P')
8 )
9
10 # Value (float)
11 self.value_label = ttk.Label(self, text="Значение*")
12 self.value_label.grid(row=4, column=0, padx=5, pady=5, sticky="e")
13 self.value_entry = EntryEx(self)
14 self.value_entry.grid(row=4, column=1, columnspan=3,
15     padx=5, pady=5, sticky="ew")
16 self.value_entry.configure(
17     validate="key",
18     validatecommand=(self.register(self._validate_float), '%P')
19 )
20
21 # Delta (float)
22 self.delta_label = ttk.Label(self, text="Абс. погрешность")
23 self.delta_label.grid(row=5, column=0, padx=5, pady=5, sticky="e")
24 self.delta_entry = EntryEx(self)
25 self.delta_entry.grid(row=5, column=1, padx=5, pady=5, sticky="ew")
26 self.delta_entry.configure(
27     validate="key",
28     validatecommand=(self.register(self._validate_float), '%P')
29 )
30
31 # Epsilon (float)
32 self.epsilon_label = ttk.Label(self, text="Отн. погрешность")
33 self.epsilon_label.grid(row=5, column=2, padx=5, pady=5, sticky="e")
34 self.epsilon_entry = EntryEx(self)
35 self.epsilon_entry.grid(row=5, column=3, padx=5, pady=5, sticky="ew")
36 self.epsilon_entry.configure(
37     validate="key",
38     validatecommand=(self.register(self._validate_float), '%P')
39 )
40
41 # Dim (float)
42 self.dim_label = ttk.Label(self, text="Множитель")
43 self.dim_label.grid(row=6, column=0, padx=5, pady=5, sticky="e")
44 self.dim_entry = EntryEx(self)
45 self.dim_entry.grid(row=6, column=1, padx=5, pady=5, sticky="ew")
46 self.dim_entry.configure(
47     validate="key",
48     validatecommand=(self.register(self._validate_float), '%P')
49 )

```

Рисунок 48 — Код в файле «MeasForm.py» часть 2

```

1 # Checkbox is_direct (bool)
2 self.is_direct_var = BooleanVar(value=True)
3 self.is_direct_check = ttk.Checkbutton(
4     self,
5     text="Прямое измерение",
6     variable=self.is_direct_var,
7     onvalue=True,
8     offvalue=False
9 )
10 self.is_direct_check.grid(row=6, column=3, padx=5, pady=5, sticky="w")
11
12 # Кнопки
13 self.add_button = ttk.Button(self, text="Сохранить",
14                             command=self.apply)
15 self.add_button.grid(row=7, column=1, padx=5, pady=5, sticky="ew")
16
17 self.clear_button = ttk.Button(self, text="Отмена",
18                                command=self.clear_form)
19 self.clear_button.grid(row=7, column=2, padx=5, pady=5, sticky="ew")
20
21 # Расширение кнопок
22 self.grid_columnconfigure(1, weight=1)
23 self.grid_columnconfigure(3, weight=1)
24
25 def _validate_float(self, value: str):
26     """ Валидация ввода для float значений """
27     return float_re.match(value) is not None or value in ["", "-"]
28
29 def clear_form(self):
30     """ Очистка/сброс всех полей ввода """
31     self.name_entry.delete(0, END)
32     self.unit_entry.delete(0, END)
33     self.char_entry.delete(0, END)
34     self.value_entry.delete(0, END)
35     self.delta_entry.delete(0, END)
36     self.epsilon_entry.delete(0, END)
37     self.dim_entry.delete(0, END)
38     self.is_direct_var.set(True)
39
40 # Заполнение полей в случае редактирования
41 if self.current_meas is not None:
42     self._fill(self.current_meas)
43
44 def _fill(self, meas: Measurement):
45     """ Заполнение форм """
46     self.name_entry.insert(0, meas.name)
47     self.unit_entry.insert(0, "" if meas.unit is None else meas.unit)
48     self.char_entry.insert(0, "" if meas.char is None else meas.char)
49     self.value_entry.insert(0, meas.value)
50     self.delta_entry.insert(0, meas.delta)

```

Рисунок 49 — Код в файле «MeasForm.py» часть 3


```

1 self.epsilon_entry.insert(0, meas.epsilon)
2 self.dim_entry.configure(state=DISABLED)
3 self.is_direct_var.set(meas.is_direct)
4
5 def apply(self):
6     """ Обработка нажатия кнопки сохранения """
7     try:
8         name = self.name_entry.get()
9         if name.replace(' ', '') == '':
10             raise Exception("Имя обязательно!")
11
12         value = self.value_entry.get()
13         if value in ['', '-']:
14             raise Exception("Значение обязательно!")
15
16         delta = self.delta_entry.get()
17         epsilon = self.epsilon_entry.get()
18         dim = self.dim_entry.get()
19         delta = None if delta in ['', '-'] else float(delta)
20         epsilon = None if epsilon in ['', '-'] else float(epsilon)
21         dim = None if dim in ['', '-'] else float(dim)
22
23         m = Measurement(
24             float(value),
25             delta,
26             epsilon,
27             name,
28             self.char_entry.get(),
29             self.unit_entry.get(),
30             dim,
31             self.is_direct_var.get()
32         )
33         self.apply_command(self.current_meas, m)
34         if self.current_meas is not None:
35             m.mid = self.current_meas.mid
36             self.current_meas = m
37     except ValueError as e:
38         messagebox.showwarning("Ошибка", "Некорректный ввод")
39         print(e)
40     except Exception as e:
41         messagebox.showwarning("Ошибка", str(e))
42         print(e)

```

Рисунок 50 — Код в файле «MeasForm.py» часть 4

```

1 from peewee import *
2
3 # База данных
4 db = SqliteDatabase(None, pragmas={'foreign_keys': 1})
5
6 class Experiment(Model):
7     exp_id = PrimaryKeyField()
8     name = CharField()
9
10    class Meta:
11        database = db
12
13 class Meas(Model):
14     meas_id = PrimaryKeyField()
15     exp_id = ForeignKeyField(Experiment, to_field="exp_id", on_delete='cascade')
16
17     name = CharField(null=True)
18     char = CharField(null=True)
19     unit = CharField(null=True)
20     value = FloatField()
21     delta = FloatField(null=True)
22     epsilon = FloatField(null=True)
23     direct = BooleanField(default=False)
24
25    class Meta:
26        database = db
27
28 # Таблицы базы данных
29 db_tables = [Experiment, Meas]

```

Рисунок 51 — Код в файле «Models.py» часть 1

```

1 from tkinter import *
2 from tkinter import ttk, messagebox
3 from EntryEx import EntryEx
4 from typing import Callable
5
6
7 class SaveMeasFrame(ttk.LabelFrame):
8     def __init__(self, master: Misc|None,
9                 apply_command: Callable, *args, **kwargs):
10         super().__init__(master, *args, **kwargs)
11         self.apply_command = apply_command
12         self._create_widgets()
13
14     def _create_widgets(self):
15         # Name (обязательное, до 80 символов)
16         self.name_label = ttk.Label(self, text="Название*")
17         self.name_label.grid(row=1, column=0, padx=5, pady=5, sticky="e")
18         self.name_entry = EntryEx(self)
19         self.name_entry.grid(row=1, column=1, columnspan=3,
20                             padx=5, pady=5, sticky="ew")
21         self.name_entry.configure(
22             validate="key",
23             validatecommand=(self.register(lambda text: len(text) <= 80), '%P')
24         )
25
26         # Char (до 40 символов)
27         self.char_label = ttk.Label(self, text="Символ")
28         self.char_label.grid(row=2, column=0, padx=5, pady=5, sticky="e")
29         self.char_entry = EntryEx(self)
30         self.char_entry.grid(row=2, column=1, padx=5, pady=5, sticky="ew")
31         self.char_entry.configure(
32             validate="key",
33             validatecommand=(self.register(lambda text: len(text) <= 40), '%P')
34         )
35
36         # Unit (до 40 символов)
37         self.unit_label = ttk.Label(self, text="Ед. изм.")
38         self.unit_label.grid(row=2, column=2, padx=5, pady=5, sticky="e")
39         self.unit_entry = EntryEx(self)
40         self.unit_entry.grid(row=2, column=3, padx=5, pady=5, sticky="ew")
41         self.unit_entry.configure(
42             validate="key",
43             validatecommand=(self.register(lambda text: len(text) <= 40), '%P')
44         )
45
46         # Кнопки
47         self.add_button = ttk.Button(self, text="Создать",
48                                     command=self.apply)
49         self.add_button.grid(row=3, column=1, padx=5, pady=5, sticky="ew")
50

```

Рисунок 52 — Код в файле «SaveMeasFrame.py» часть 1

```

1 self.clear_button = ttk.Button(self, text="Очистить",
2                               command=self.clear_form)
3 self.clear_button.grid(row=3, column=2, padx=5, pady=5, sticky="ew")
4
5 # Расширение кнопок
6 self.grid_columnconfigure(1, weight=1)
7 self.grid_columnconfigure(3, weight=1)
8
9 def clear_form(self):
10     """ Очистка/сброс всех полей ввода """
11     self.name_entry.delete(0, END)
12     self.unit_entry.delete(0, END)
13     self.char_entry.delete(0, END)
14
15 def apply(self):
16     """ Обработка нажатия кнопки сохранения """
17     try:
18         name = self.name_entry.get()
19         if name.replace(' ', '') == '':
20             raise ValueError("Имя обязательно!")
21
22         self.apply_command(name,
23                             self.char_entry.get(),
24                             self.unit_entry.get()
25                             )
26
27     except ValueError as e:
28         messagebox.showwarning("Ошибка ввода", str(e))
29         print(e)

```

Рисунок 53 — Код в файле «SaveMeasFrame.py» часть 2

ПРИЛОЖЕНИЕ Б

Интерфейс

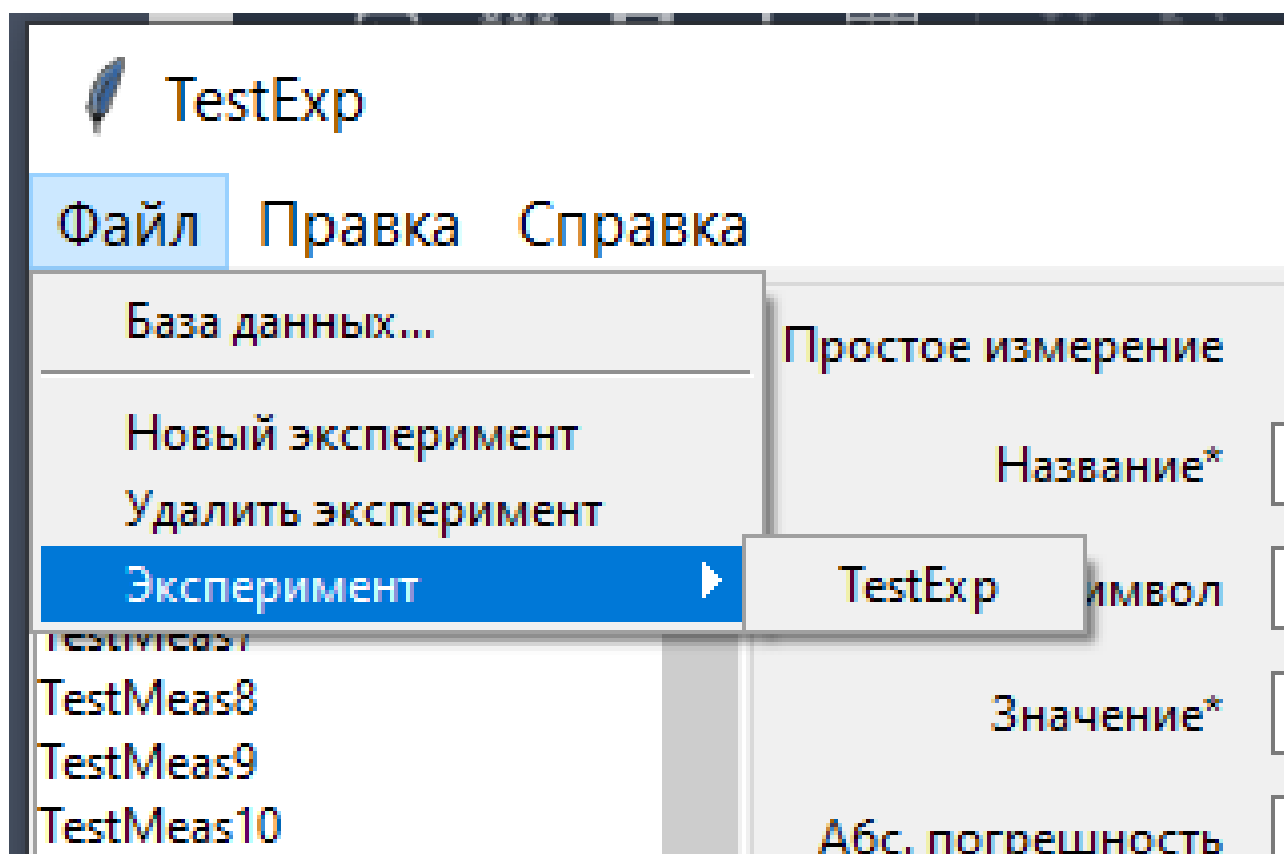


Рисунок 54 — Выбор эксперимента

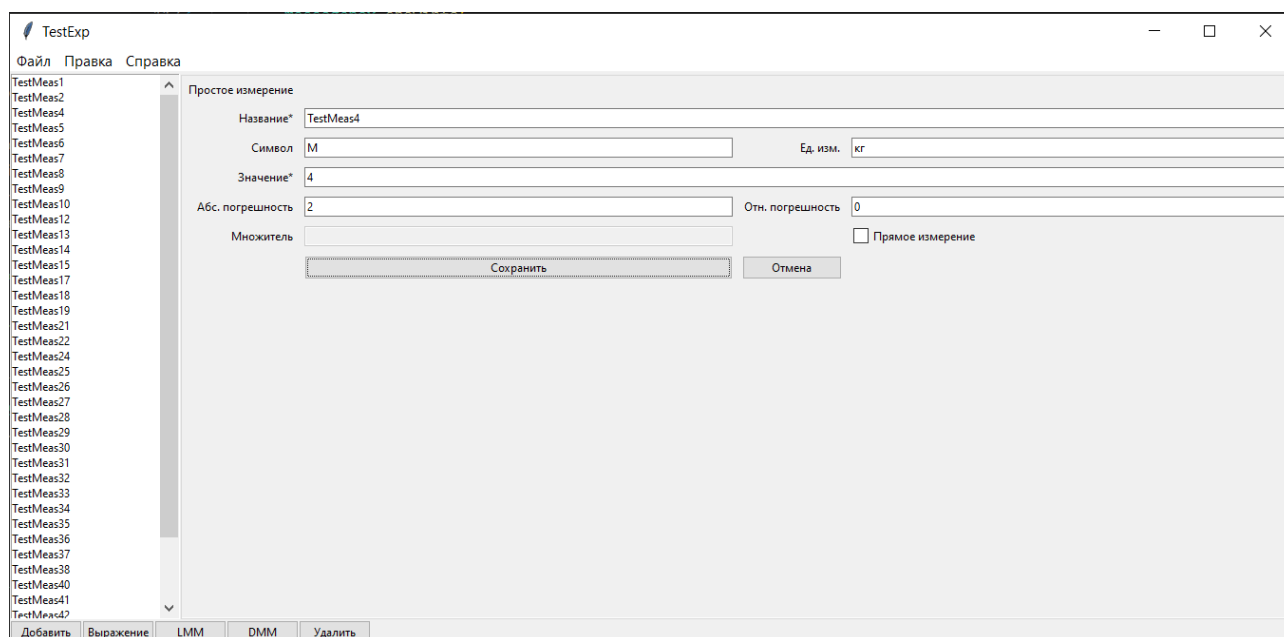


Рисунок 55 — Редактирование измерения

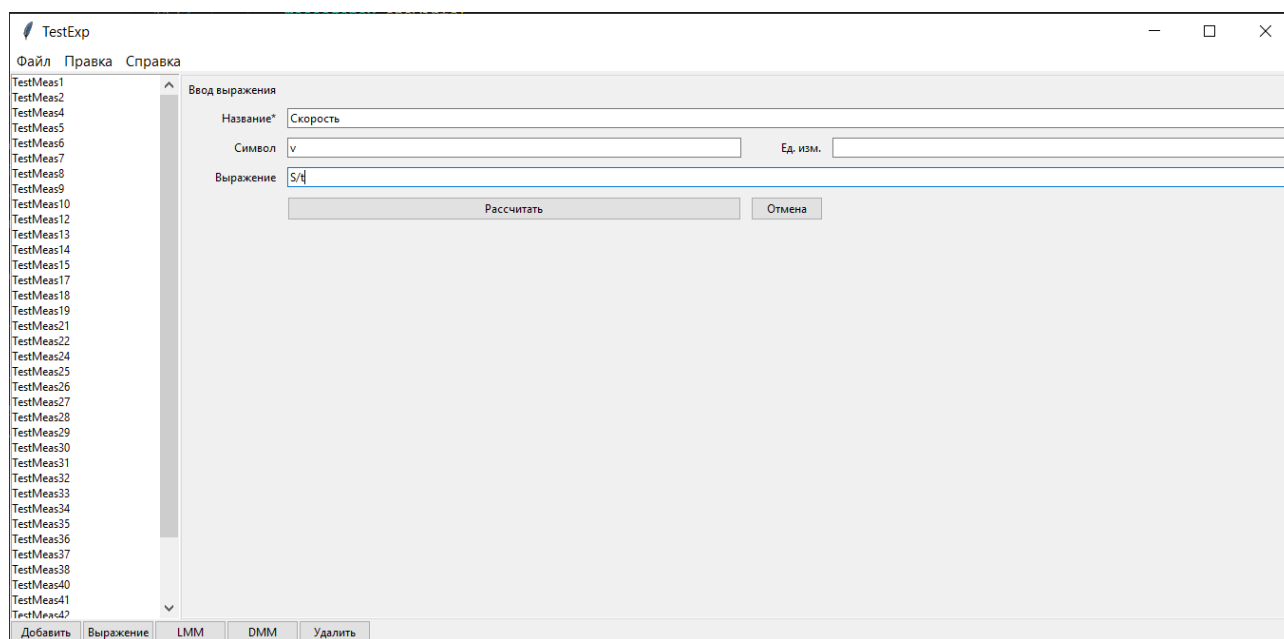


Рисунок 56 — Ввод формул

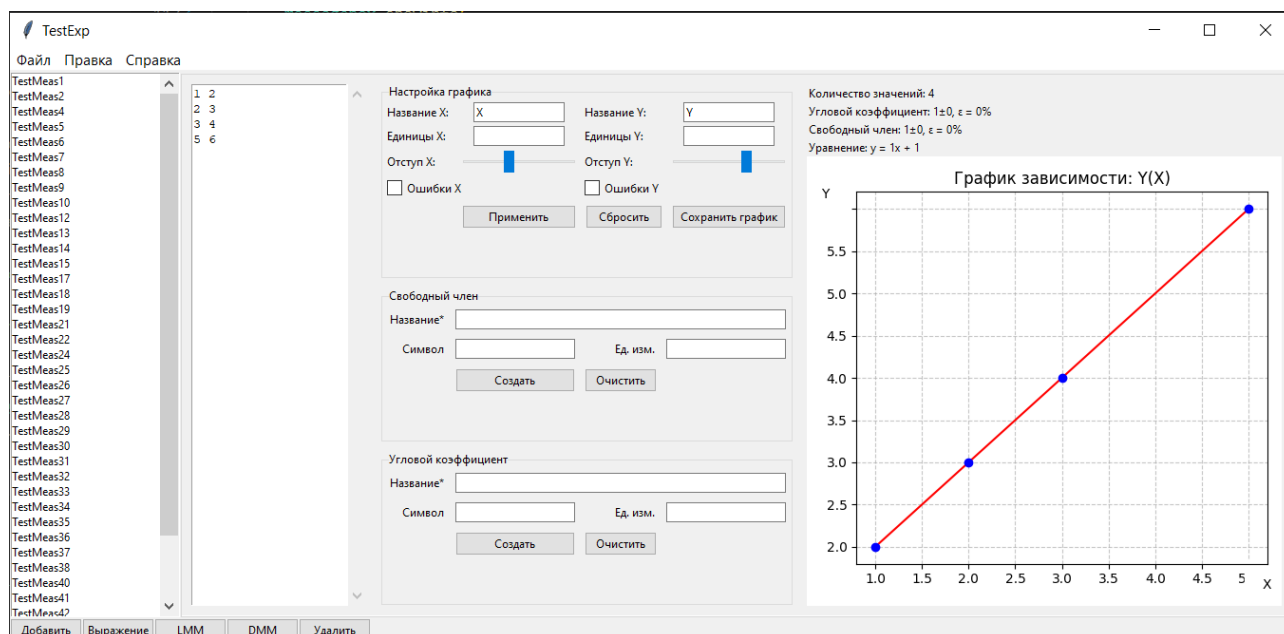


Рисунок 57 — Работа с совместными измерениями